

Today's Agenda:-

Starting 7:05

1. Max Subarray Sum
2. Continuous Sum Query from i to j .
3. Merge Overlapping Intervals.

Introduction

- Harshal Garg 🧑
- Software Development Engineer at Kotak Mahindra Bank 📁
- Ex-Amazon 🛒 | Ex-Oracle 🏛️
- B.Tech in CSE, NIT Jalandhar (2018) 🎓
- 3 years of teaching experience with 300+ DSA lectures 🧑📖
- Originally from Agra 🕌, currently in Hyderabad 🏙️

Q:- Given an integer array A of size N .
Find maximum subarray sum out of all subarrays.

Ex:- $\{-2, 3, 4, -1, 5, -10, 7\}$

11.

Ex:- $\{-3, 4, 6, 8, -10, 2, 7\}$

18.

Quiz 1

$\{4, 5, 2, 1, 6\}$

18.

Quiz 2

$\{-4, -3, -6, -9, -2\}$

ans = -2.

Brute Force:-

Consider each & every subarray.

Iterate on that subarray to find the sum. Compare that sum with global or & update G.A. if got a better sum.

T.C $\rightarrow O(N^3)$; S.C $\rightarrow O(1)$.



Prefix Sum

T.C $\rightarrow O(N^2)$; S.C $\rightarrow O(N)$



Carry Forward

T.C $\rightarrow O(N^2)$; S.C $\rightarrow O(1)$.

Optimisation

Possible cases

Case I : All elements are positive

$\{4, 2, 1, 6, 7\}$ Entire Array

Case II : All elements are negative

$\{-4, -2, -1, -6, -7\}$ Largest element

Case III : Some positives in between negatives

$\{-3, -2, -6, 4, 3, 6, 9, -5, -3, -1, -6\}$

Case IV : Positive & negative on either side.

$\{-3, -2, -6, 5, 3, 4, 6\}$

Kadane.

Generic Case

40 735

-35

{ ... +, +, +, +, -5, -10, -20, +, +, +, +, ... }

30

40

5, 20, ~~30~~, ~~25~~, ~~15~~, ~~-5~~, 0.

Kadane's Algorithm

i

EX:- { 0 1 2 3 4 5 6 7 8
-20, 10, -20, -12, 6, 5, -3, 8, -2 }

i

Cur Sum

= ~~0~~ -20 ~~10~~ -10 ~~-12~~ ~~6~~ ~~5~~ ~~-3~~ ~~8~~ ~~14~~

max Sum

= ~~-~~ ~~-20~~ ~~10~~ ~~11~~ 16

Quiz 3:-

$\{ \overset{i}{\underset{\text{index}}{\begin{matrix} -2 & 3 & 4 & -1 & 5 & -10 & 7 \end{matrix}}} \}$

Cur Sum = ~~0~~ ~~-2~~ ~~0~~ ~~3~~ ~~7~~ ~~8~~ ~~11~~ ~~8~~
max Sum = ~~-0~~ ~~-2~~ ~~3~~ ~~7~~ **11**

Code

```
int maxSubarraySum (int arr, int N) {
```

```
    int curSum = 0, ans = -0;
```

```
    for (i = 0; i < N; i++) {
```

```
        # Add C.V. to curSum
```

```
        curSum += arr[i];
```

```
        # Compare with ans
```

```
        ans = max(ans, curSum);
```

```
        # Reset if needed.
```

```
        curSum = max(0, curSum);
```

```
    }
```

```
    return ans;
```

T.C $\rightarrow O(N)$

S.C $\rightarrow O(1)$

}

2. Given an integer array A where every element is 0. Return the final array after performing multiple queries.

Query (i, x) : Add x to all elements

Q-2: given a integer array A where every element is 0 . we have to return the final array after performing multiple queries on it

Query=> query(i,x): add x to all the elements from i to n-i.

Queries

	0	1	2	3	4	5	6
A = {	0	0	0	0	0	0	0
		+3	3	3	3	3	3

(1, 3) ✓
(4, -2) ✓
(3, 1)

-2 -2 -2
+1 +1 +1 +1

A = {0, 3, 3, 4, 2, 2, 2}

Quiz 4

	0	1	2	3	4
A = {	0	0	0	0	0
		+3	+3	+3	+3

Queries

+2 +2 +2 +2 +2
+1

(1, 3)
(0, 2)
(4, 1)

A = {2, 5, 5, 5, 6}

Brute Force

For each query, traverse the array to update the value.

$$T.C \rightarrow O(Q \times N)$$

Optimisation

$$A = \overset{0}{\{0, \overset{1}{0}, \overset{2}{0}, \overset{3}{0}, \overset{4}{0}\}}$$

+3
+2
+1

Queries

(1, 3)
(0, 2)
(4, 1)

$$A = \{2, 3, 0, 0, 1\}$$



Apply pSum

$$A = \{2, 5, 5, 5, 6\}$$

Code.

	0	1
0	0	2
1	1	3
2	2	6
3	0	4
4	1	9

Q →

```
for (i = 0 ; i < Q ; i++) {  
    index = Q[i][0];  
    value = Q[i][1];  
    arr[index] += value;  
}
```

3

Q

```
N {  
    for (i = 1 ; i < N ; i++) {  
        arr[i] = arr[i-1] + arr[i];  
    }  
}
```

return arr;

T.C → $O(Q + N)$.
S.C → $O(1)$

Variation

Query $\longrightarrow (i, j, x)$
Add element x to all
the indices b/w i to j .

$\nearrow j+1$

0	1	2	3	4	5	6
0	0	0	0	0	0	0
	+2	+2	+2	+2	+2	+2
				-2	-2	-2

Queries

$(1, 3, 2) \nearrow (1, 2)$
 $(2, 5, 3) \nwarrow (2, 3)$
 $(5, 6, -1) \nearrow (5, -1)$
 $(3+1, -2)$
 $(5+1, -3)$
 $(6+1, +1) \times$

+3	+3	+3	+3	+3	
				-3	
				-1	-1

$A = [0, 2, 5, 5, 3, 2, -1]$

Quiz 5

0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0
	+3	+3	+3	+3			

i, j, x

-1	-1	-1	-1	-1	-1
		4			

1	4	3			
0	5	-1			
2	2	4			
4	6	3			

$A = [-1, 2, 6, 2, 5, 2, 3, 0]$

Code:

Q

```
for (i = 0 ; i < Q ; i++) {  
    i = Q[i][0];  
    j = Q[i][1];  
    value = Q[i][2];  
  
    arr[i] += value;  
    if (j+1 < N) {  
        arr[j+1] += -value;  
    }  
}
```

3 3

N

```
for (i = 1 ; i < N ; i++) {  
    arr[i] = arr[i-1] + arr[i];  
}
```

T.C $\rightarrow O(Q+N)$.

S.C $\rightarrow O(1)$.

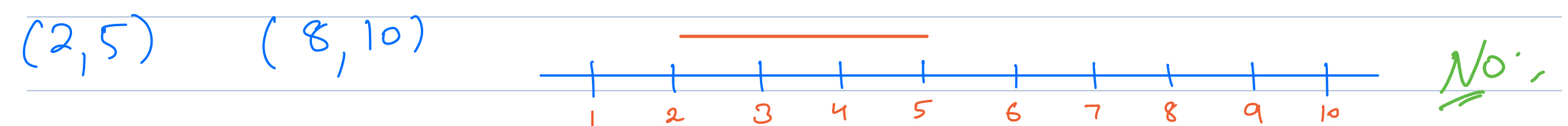
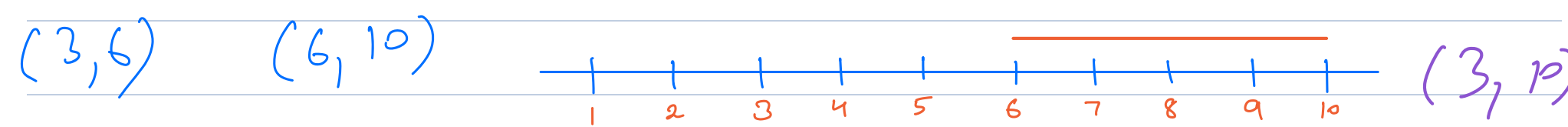
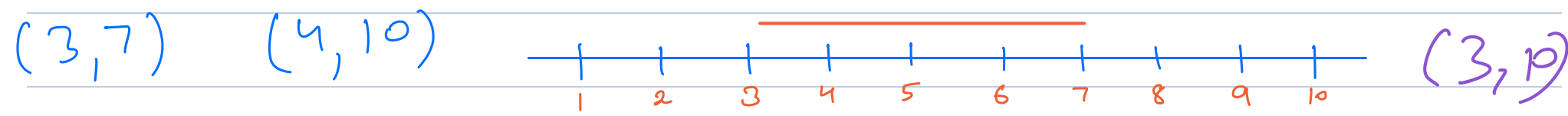
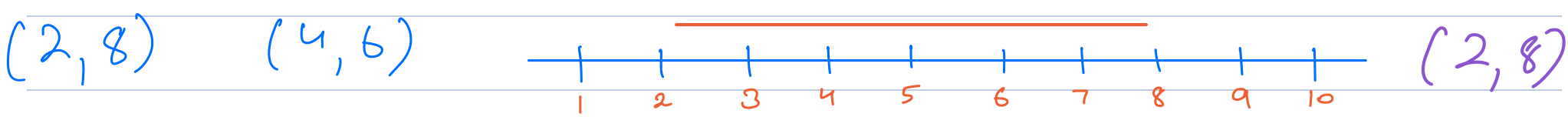
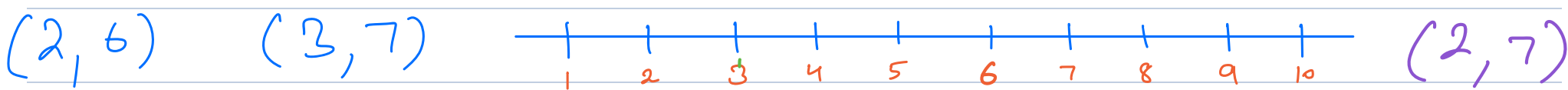
8:40.

Merge Intervals

An interval is defined by start & end time.

I_1 I_2

s e



Non-overlapping Condition

(s_1, e_1)
 (s_2, e_2)

s_1 e_1



s_2 e_2

$$(e_1 < s_2 \quad || \quad e_2 < s_1)$$

Non Overlapping
Condition..

Q:-6

$$\begin{matrix} s_1 & e_1 \\ (3, & 8) \end{matrix} \quad \& \quad \begin{matrix} s_2 & e_2 \\ (5, & 12) \end{matrix}$$

$$(\cancel{8 < 5} \quad || \quad \cancel{12 < 3}).$$

false

Overlap

Q:-7

$$\begin{matrix} s_1 & e_1 \\ (6, & 10) \end{matrix}$$

$$\begin{matrix} s_2 & e_2 \\ (8, & 15) \end{matrix}$$

$$(e_1 < s_2 \quad || \quad e_2 < s_1)$$

Overlap

$$(\cancel{10 < 8} \quad || \quad \cancel{15 < 6})$$

false

$$s =$$

$$\min(s_1, s_2)$$

$$e =$$

$$\max(e_1, e_2)$$

$$\underline{[6, 15]}$$

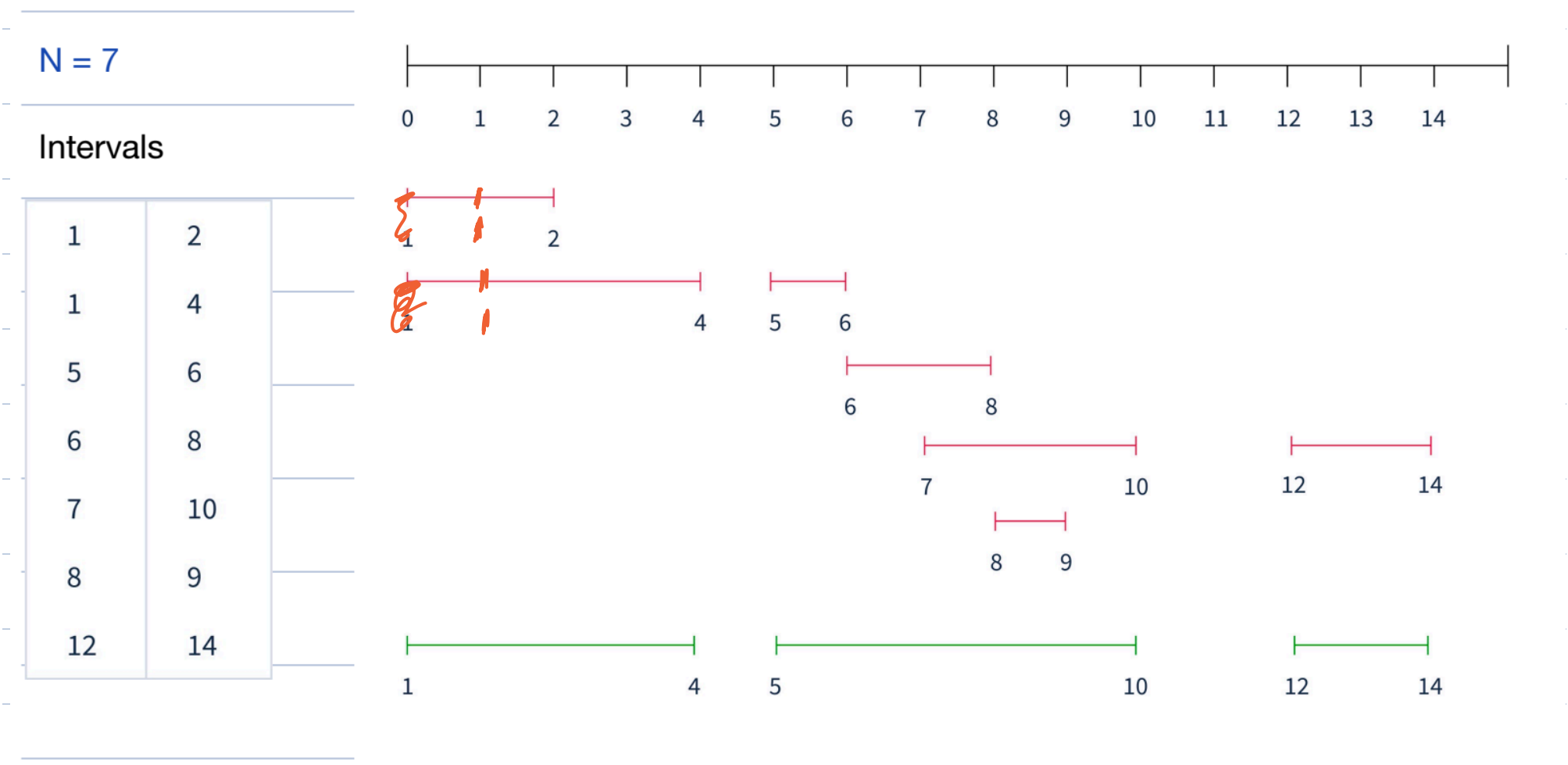
Q-6 :-

Q-7:

3. You are given a collection of intervals
A in a 2D format. $[start, end]$.
Intervals are sorted based on start values.

Merge all overlapping intervals &
return the resulting set of non
overlapping intervals.

Ex:- $[\{1, 2\}, \{1, 4\}, \{5, 6\}, \{6, 8\}$
 $, \{7, 10\}, \{8, 9\}, \{12, 14\}]$



Ans = $[[1, 4], [5, 10], [12, 14]]$.

Q-8.

$[\{1, 10\}, \{2, 3\}, \{4, 5\}, \{9, 12\}]$

I_1	I_2	<u>Overlapping</u>	<u>Ans. List</u>
$\{1, 10\}$	$\{2, 3\}$	✓	—
$\{1, 10\}$	$\{4, 5\}$	✓	—
$\{1, 10\}$	$\{9, 12\}$	✓	—
$\{1, 12\}$			
Ans :- <u>$\{1, 12\}$</u>			

Approach

1. Create an empty array to store the intervals.

2. Take an array as cum. & start indexing from 1st interval onwards.

If i^{th} interval overlaps:

Update cum. after merging.

$e_1 < s_2$

else :-

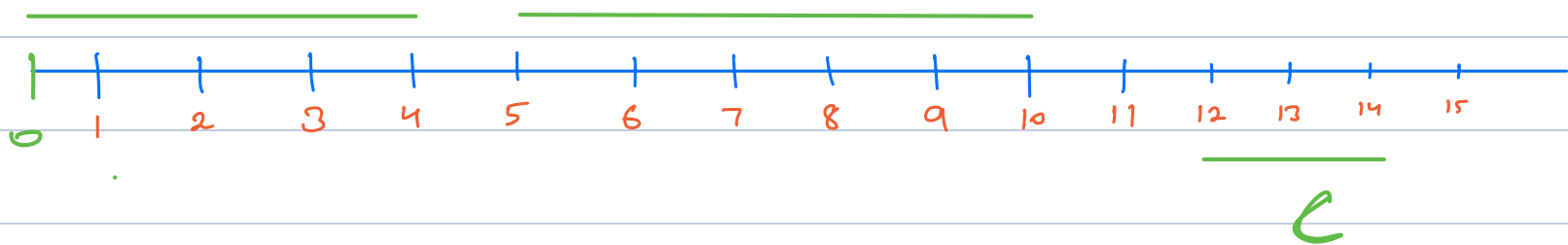
Store cum. in ans; &

reset cum. to i^{th} .

3. Return ans;

Dry Run

[{0, 2}, {1, 4}, {5, 6}, {6, 8},
 , {7, 10}, {8, 9}, {12, 14}]
i



<u>Cur</u>	<u>ith</u>	is (e1 > s2) End of cur > Start of i th	<u>After Merge.</u>	<u>Ans. List</u>
(0, 2)	(1, 4)	True.	(0, 4)	—
(0, 4)	(5, 6)	False.	—	(0, 4)
(5, 6)	(6, 8)	True.	(5, 8)	(0, 4)
(5, 8)	(7, 10)	True	(5, 10)	(0, 4)
(5, 10)	(8, 9)	True.	(5, 10)	(0, 4)
(5, 10)	(12, 14)	False	—	(0, 4), (5, 10)
(12, 14)	—	—	—	(0, 4), (5, 10) & (12, 14).

$$\underline{currS \leq IS}$$

Code

```
currS = arr[0][0];
currE = arr[0][1];
```

```
for (i=1; i < N; i++) {
```

```
    iS = arr[i][0];
```

```
    iE = arr[i][1];
```

Overlapping Condition.

```
    if (currE > iS) {
```

Redundant.

```
        currS = min(currS, iS);
```

```
        currE = max(currE, iE);
```

}

```
    else {
```

```
        ans.add({currS, currE});
```

```
        currS = iS;
```

```
        currE = iE;
```

}

}

```
ans.add({currS, currE});
```

last interval

handled

return ans;

T.C $\rightarrow O(N)$

S.C $\rightarrow O(1)$

Real Life Application

Background

Scaler Academy, a leading ed-tech platform known for its comprehensive learning programs, is planning to conduct maintenance on its website to enhance user experience and introduce new features.

To ensure the maintenance work does not disrupt the learning process for its students, Scaler Academy aims to schedule this maintenance during the period of **no user activity**.

Problem Statement

Given sorted data on the active hours of multiple learners on the platform, your task is to analyze this data and identify the longest continuous period when no learners are active. This identified time slot is crucial as it represents the best opportunity to perform website maintenance with the least disruption to learners' activities.

Approach a Problem

